

The goals of my customer Churn project is for me to do indepth EDA in python following all the standard procedures and then switch from an exploratory to a modeling mindset and also to become familiar with the key modeling terms and understand the different types of ways that data needs to be structured to be input into machine learning models. And finally, I want to become familiar with the most common feature engineering techniques.

1. Scope the Project

The primary objective of this project is to use a supervised learning technique **to predict which customers are most likely to cancel their subscription.**

To do this , I used three months of customer data, focusing specifically on their subscription details and listening habits.

2. Gather Data

To begin the analysis, I read several data sources into Python using the **Pandas Libraries.**

The data is stored in two main files:

- **maven_music_customers.csv:** This contains basic customer information, such as how long they have been members, their subscription plans, and their cancellation dates.
- **maven_music_listening_history.xlsx:** This is an Excel file with multiple tabs (sheets) that contain more granular data

Reading the csv: use `pd.read_csv()` to bring in the customer table.

Reading Multiple Excel Sheets: Because the listening history file has several tabs, I use `pd.read_excel()` and specify the `sheet_name` to extract three distinct DataFrames:

- **Listening History (Sheet 1):** Provides IDs for customers, sessions, and the audio they heard.
- **Audio Data (Sheet 2):** Contains details about the specific songs and podcasts, including their names, genres, and popularity scores.
- **Session Data (Sheet 3):** Lists every session ID along with the specific login times.

3. Data Cleaning

In the **Data Cleaning** stage, I transition from raw data to a format suitable for analysis.

This process involves five critical sub-steps to ensure the accuracy and usability of my customer churn model.

1. Convert Data Types

When initially loaded, many columns in the customer's DataFrame are read as generic "objects" (strings). To perform calculations, I must convert them:

- **Dates:** Use `pd.to_datetime()` for the Member Since and Cancellation Date columns.
- **Subscription Rate:** This column contains dollar signs (e.g., "\$2.99"). So I used `.str.replace('$', '')` to remove the symbol before using `pd.to_numeric()` to convert it into a float.

2. Resolve Missing Data (NaNs)

Next I checked for null values there were three columns with null values in the customers table:

- **Subscription Plan:** Several entries are missing. By analyzing unique rates, I discovered that every customer paying \$2.99 is on the "Basic (Ads)" plan. Therefore, missing plans are filled with "Basic (Ads)" using `.fillna()`.
- **Discount:** This column only contains "Yes" or NaN. The logic applied is that a NaN means the customer did *not* receive a discount. These are converted to 1s (Yes) and 0s (No) using `np.where()`.
- **Cancellation Date:** NaNs here represent active customers who have not canceled yet, so they are left as "Not a Time" (NaT) for now.

3. Fix Typos and Inconsistent Text

Clean data requires uniform categories and realistic values:

- **The 99.99 Typo:** Using `.describe()`, I found a maximum subscription rate of \$99.99, which is far above the \$9.99 Premium plan. This was identified as a typo and corrected to 9.99 using `.iloc`.
- **Genre Unification:** In the audio table, both "Pop" and "Pop Music" existed as separate genres. I unified these into a single "Pop" category using `np.where()`.

4. Remove Duplicates and Outliers

Then I checked all four tables for duplicate rows using `.duplicated()`, but none were found. The `.describe()` method is used again to verify that all numeric ranges (like Customer IDs and popularity scores) are reasonable and free of extreme outliers.

5. Feature Creation (New Columns)

Finally, I create two new columns that are essential for the upcoming EDA and modeling:

- **Canceled:** A binary column where 1 represents a canceled subscription and 0 represents an active one. This is derived by checking if the Cancellation Date is "not null" using `.notna()`.
- **Clean Email:** The raw email column includes a prefix like "Email: ". The code uses string slicing (`.str[6:]`) to remove this prefix and keep only the actual email address

4. Exploratory Data Analysis (EDA)

The focus shifts from cleaning the data to uncovering patterns that explain why customers are leaving. The analysis is divided into two main parts: understanding customer characteristics and analyzing their listening behavior.

1. Analyzing Canceled Customers

The first goal is to understand the profile of those who churned by looking at two specific metrics:

- **Average Membership Duration:** By subtracting the Member Since date from the Cancellation Date for churned users, the analysis found that customers were members for an average of only **46 days**.
 - **Insight:** This duration is considered very low (roughly 1.5 months). However, the sources note this might be skewed because the project only uses three months of data, which limits the view of long-term customers.
- **The "Discount Effect":** The analysis compared the cancellation rates of people who received discounts versus those who did not.
 - **Result:** **85%** of customers with a discount canceled, while only **30%** of those without a discount canceled.
 - **Insight:** This suggests that discount-seekers may not be loyal long-term users and are likely to leave once the promotion ends.

2. Deep Dive into Listening History

To understand how usage habits affect churn, I write a code joining the **Listening History** and **Audio** tables. This requires some technical maneuvering:

- **Data Preparation (The ID Split):** The ID in the audio table was formatted as a string (e.g., "Song-101"), but the listening history used a numeric ID. To join them, the code used `str.split(' - ')` to extract the number and then converted it to an **integer** to match the other table.
- **Customer Session Volume:** Using `groupby('Customer ID')` and counting unique `Session IDs`, the analysis visualized the distribution of listening activity.
 - **Insight:** Most customers logged in for 2 or 3 sessions over the three-month period, though a few "extreme listeners" had many more.
- **Genre Preferences:** By running `value_counts()` on the genres, it was discovered that **Pop** is the most popular category by a significant margin, while True Crime podcasts have the lowest engagement

5. Feature Engineering

The goal is to prepare the data for predictive modeling by creating a clean, numeric, and non-null DataFrame where each row represents a single customer. The "Model DataFrame" (`model_df`) will hold the specific predictors that might influence a customer's decision to cancel.

1. Initializing the Model DataFrame

I start by subsetting the cleaned `customers` table to include three essential columns I already prepared,:

- `Customer ID`: To keep track of each individual.
- `Canceled`: Our target variable (1 for canceled, 0 for active).
- `Discount?`: A binary predictor (1 for Yes, 0 for No).

2. Feature: Number of Listening Sessions

To measure engagement, I calculate how many times each customer logged in over the three-month period.

- **Logic:** Using the joined listening history, I **group by** Customer ID and use the `.nunique()` function on the `Session ID` column.
- **Integration:** I rename this count to `Number of Sessions` and merge it back into my `model_df` using a left join on Customer ID.

3. Feature: Content Preferences (Percentages)

Looking into the dataset I saw **Percent Pop** and **Percent Podcast** as potentially strong predictors. Calculating these requires several steps to turn text genres into numbers:

- **One-Hot Encoding:** I use `pd.get_dummies()` on the `Genre` column to create binary columns for every individual genre (Pop, Comedy, Hip Hop, etc.).
- **Aggregating Genre Counts:** then group these dummy variables by `Customer ID` and use `.sum()` to find the total number of items a customer listened to in each specific category,,.
- **Total Audio Volume:** To get a denominator for my percentage, I group the listening history by `Customer ID` and use `.count()` on the rows to find the total items (songs + podcasts) consumed by each user.
- **The Math:**
 - **Percent Pop:** $(\text{Pop Count} / \text{Total Audio}) * 100$,.
 - **Percent Podcast:** $((\text{Comedy Count} + \text{True Crime Count}) / \text{Total Audio}) * 100$. (Comedy and True Crime are summed because they represent the platform's podcast offerings),.

4. Final Interpretation (Correlation Analysis)

Once the `model_df` is complete with all five features, I used `.corr()` and a Seaborn **pair plot** to see which factors actually drive churn.

The Key Findings:

- **Discount (Positive Correlation):** Getting a discount is strongly linked to canceling, confirming my earlier EDA that discount-seekers are less loyal.
- **Sessions (Negative Correlation):** As the number of sessions increases, the likelihood of canceling decreases.
Active users stay.
- **Percent Pop (Negative Correlation):** Pop music fans have a high affinity for the service and are much less likely to churn.
- **Percent Podcast (No Correlation):** Whether a user listens to podcasts or not seems to have almost no impact on their decision to cancel

6: Visualizing and Interpreting results

In the final step of the project, I analyze the `model_df` to identify which specific factors are the strongest predictors of customer churn. This step moves beyond just preparing data to actually deriving actionable business insights.

1. Visualizing with Pair Plots

To get a high-level view of how all my engineered features relate to one another and to the cancellation status, the code uses the **Seaborn library**.

- **The Code:** `sns.pairplot(model_df)`.
- **The Purpose:** A pair plot creates a grid of scatter plots and histograms, allowing me to see patterns across all variables at once.
- **The Observation:** The sources note that because this specific project uses a relatively small dataset (30 customers), the visual scatter plots in the pair plot can be difficult to interpret for clear trends.

2. Interpreting with Correlation Coefficients

To gain more precision than the visual plot provided, the analysis uses the `.corr()` **method and heatmap**. This generates a correlation matrix that quantifies the strength of the relationship between the features and the target variable (`Canceled`).

3. Key Insights from the Final Model

The correlation analysis revealed four major findings that would be the "stars" of my portfolio project:

- **Discount (Positive Correlation):** There is a clear positive correlation between receiving a discount and canceling. This confirms that customers who join for a promotion are significantly more likely to churn once that promotion ends.
- **Number of Sessions (Negative Correlation):** This is one of the strongest predictors. The more frequently a customer logs in, the less likely they are to cancel. **Active users are loyal users.**
- **Percent Pop (Strong Negative Correlation):** Fans of Pop music show a high affinity for the service. A higher percentage of Pop music in their listening history is a strong indicator that they will remain a subscriber.
- **Percent Podcast (No Correlation):** Interestingly, whether a customer listens to podcasts or not has almost no relationship with their decision to cancel. This suggests that podcast content is currently "churn-neutral" for Maven Music